

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P1958R0
Date:	2020-01-12
Audience:	LEWG
Revises:	P0260R3
Author:	Lawrence Crawl, Chris Mysen
Contact:	Lawrence@Crawl.org

C++ Concurrent Buffer Queue

Lawrence Crawl, Chris Mysen

Abstract

Concurrent queues are a fundamental structuring tool for concurrent programs. We propose a concrete concurrent queue, based on a fixed-size buffer, that meets the concepts described in [P0260R4](#).

Contents

Introduction

Queues provide a mechanism for communicating data between components of a system.

We propose a concrete concurrent queue, based on a fixed-size buffer, that meets the concepts described in [P0260R4](#).

Locking Buffer Queue

We provide a concrete concurrent queue in the form of a fixed-size `buffer_queue`. It meets the `waitingConcurrentQueue` concept. It provides for construction of an empty queue, and construction of a queue from a pair of iterators. Constructors take a parameter specifying the maximum number of elements in the buffer. Constructors may also take a parameter specifying the name of the queue. If the name is not present, it defaults to the empty string.

The `buffer_queue` deletes the default constructor, the copy constructor, and the copy assignment operator. We feel that their benefit might not justify their potential confusion.

Proposed Wording

The concurrent queue container definition is as follows. The section, paragraph, and table references are based on those of [N4567](#) *Working Draft, Standard for Programming Language C++*, Richard Smith, November 2015.

?.? Concurrent queues [conqueues]

Section added by [P0260R4](#).

Section added by [P0260R4](#).

?.?.5 Concrete queues [conqueues.concrete]

Add a new section:

There is one concrete concurrent queue type.

?.?.5.1 Class template `buffer_queue`[conqueues.concrete.buffer]

Add a new section:

```
template <typename Value>  
class buffer_queue  
{  
    buffer_queue() = delete;  
    buffer_queue(const buffer_queue&) = delete;  
    buffer_queue& operator =(const buffer_queue&) = delete;  
public:  
    typedef Value value_type;  
    explicit buffer_queue(size t max_elems);  
    template <typename Iter>  
    buffer_queue(size t max_elems, Iter first, Iter last);  
    ~buffer_queue() noexcept;  
  
    void close() noexcept;  
    bool is_closed() const noexcept;  
    bool is_empty() const noexcept;  
    bool is_full() const noexcept;  
    static bool is_lock_free() noexcept;  
  
    Value value_pop();  
    queue_op_status wait_pop(Value&);  
    queue_op_status try_pop(Value&);  
  
    void push(const Value& x);  
    queue_op_status wait_push(const Value& x);  
    queue_op_status try_push(const Value& x);  
    void push(Value&& x);  
    queue_op_status wait_push(Value&& x);  
    queue_op_status try_push(Value&& x);  
};
```

The class template `buffer_queue` implements the `waitingConcurrentQueue` concept. The class template `buffer_queue` provides a FIFO ordering. That is, the sequentially consistent order of pushes is the same as the sequentially consistet order of pops. (See the synchronization of push [conqueues.concept.wait].)

`buffer_queue(size t max_elems);`

Effects: Constructs the queue with a buffer capacity of `max_elems`.

Throws: Any exception thrown by constructing the internal arrays or mutexes.

`template <typename Iter> buffer_queue(size t max_elems, Iter first, Iter last);`

Effects: Constructs the queue with a buffer capacity of max_elems. Initializes the internal array by iterating from first to last.

Throws: Any exception thrown by constructing the internal arrays or mutexes.

~buffer_queue().noexcept;

Effects: Destroys the queue. Any elements remaining within the queue are discarded.

Revision History

This paper revises portions of [P0260R3](#).